CodeVedha An Auditable Programming Assessment Platform with Correctness Bounded Scoring

A. Manigreeva[1], D. Nayaneesh[2], Kantam Pavan Sai Reddy[3], Kura Vignesh Reddy[4]*, Vitta Karthikeya[5]

Department of Computer Science and Engineering
Keshav Memorial Institute of Technology
3-5-1026, Narayanguda, Hyderabad, Telangana 500029, India

1 a.manigreeva@gmail.com; 2 nayaneesh6607@gmail.com

3 kantampavansaireddy30@gmail.com; 4 kvigneshreddy963@gmail.com

5 vittakarthikeya@gmail.com

*Corresponding author: Kura Vignesh Reddy

Abstract—Programming assessment must distinguish observed correctness, inferred resource complexity, and generated advice. CodeVedha combines isolated execution, structural analysis, deterministic style checks, and optional language model assistance under a versioned scoring contract. Its correctness ceiling limits the final grade by the fraction of evaluated cases passed, while missing weighted evidence remains explicit. A separate projection route generates a private candidate, reviews its alignment with displayed guidance, executes it, and measures its score using the saved rubric. We establish the ceiling invariant and compare four scoring rules on the same eighty-four component and weight configurations. Weighted aggregation alone exceeds observed correctness in forty-nine configurations; a fixed failure cap does so in forty-three. Both multiplicative scaling and the proposed ceiling have no unrounded violations, with different partial-credit behavior. Eight live execution contracts meet their expected outcomes, and candidate scores match fresh official scores under two weight configurations. A twenty-snippet AI-assisted pilot across five languages gives nineteen joint exact complexity-label matches with model proposals and identifies a symbolic-bound limitation. These results characterize policy behavior and implementation consistency. They do not establish expert grading accuracy, optimal weights, or educational benefit.

Index Terms—programming assessment, algorithmic complexity, automated feedback, correctness ceiling, verified score projection, reproducibility

I. Introduction

Automated execution provides a practical foundation for programming assessment, but a passed test suite reveals only behavior on the inputs that were evaluated. A resource estimate describes a different property: how work or storage changes with input size. Readability findings and generated advice introduce further judgments. A useful report should explain how these forms of evidence contribute to the grade and which conclusions remain uncertain. Earlier reviews of test-based assessment and automated programming feedback motivate examining these concerns together [1], [2].

A complexity-focused course makes this separation particularly important. A fast program that returns incorrect answers should not receive unrestricted performance credit in its final mark. Conversely, increasing the correctness weight alone does not specify what should happen when performance evidence is unavailable. Generated revisions create another consistency problem: an anticipated gain is misleading if it comes from an assumed improvement, a different rubric, or an untested replacement algorithm.

CodeVedha addresses these problems through explicit evidence boundaries. Students run public examples before submission; the server independently checks the submitted source before evaluating the full test suite. The report stores the execution result, complexity evidence, component scores, and rubric snapshot. Optional language model agents provide explanations and staged guidance, while correctness and score aggregation remain governed by explicit code. Instructor functions include assignments, enrollment, analytics, and review of similarity evidence.

The contribution is an implemented evidence contract linking three decisions: a correctness ceiling on the mark, explicit incompleteness when weighted evidence is missing, and a separately measured candidate projection tied to the saved rubric. We investigate whether these decisions prevent violations of the stipulated ceiling, how they differ from alternative correctness constraints, and whether candidate scoring agrees with a fresh official assessment. The contribution is this integration and its controlled characterization, rather than a new asymptotic inference algorithm. The arithmetic invariant alone does not establish pedagogical validity.

II. Related Work

Douce, Livingstone, and Orwell review automatic test-based programming assessment [1]. Keuning, Jeuring, and Heeren examine automated feedback generation for programming exercises [2]. These studies motivate distinguishing a verdict from the explanation that helps a learner revise a program. Piech et al. investigate learned program representations for propagating instructor feedback [3], illustrating a different route to scaling feedback from examples.

Autograder+ combines adapted language models, instructional prompt selection, and learned representations for feedback and cohort visualization [4]. Its evaluation emphasizes feedback alignment and submission representations. CodeVedha instead studies the contract connecting execution evidence, withheld scores, rubric snapshots, and revision projections. We use Autograder+ as motivation, without inheriting its data, training results, or numerical feedback claims. The present controlled

comparison varies scoring rules within CodeVedha; it is not a head-to-head evaluation of the two platforms.

APPS evaluates generated programs through programming challenges and test execution [5]. HumanEval similarly evaluates functional behavior of generated code [6]. These benchmarks inform the need to execute candidate programs instead of treating plausible source text as sufficient evidence. The present study is smaller and serves a different purpose: it tests implementation contracts and explicitly reports a synthetic, AI-assisted label-agreement pilot. Benchmark pass rates, human feedback agreement, and our regression counts are not interchangeable metrics.

III. Platform Architecture and Evidence Flow

A. Assessment Workflow

Figure 1 separates the official assessment route from the optional projection route. A React frontend communicates with a FastAPI backend, which persists accounts, assignment context, submissions, and assessment artifacts in PostgreSQL. The local deployment uses Docker Compose and a separate execution service. Supported execution languages are Python, C, C++, Java, and JavaScript. Source analysis, style assessment, aggregation, and optional model-assisted explanation are organized as specialized workflow steps; this modular organization does not itself establish superiority over a single model.

The public run action compiles or interprets the current source and displays example verdicts. Submission eligibility is rechecked on the server against the exact source in the request. A browser flag or an earlier successful run cannot authorize different code. Full assessment then evaluates the configured public and private cases. Passing every evaluated case determines acceptance; an infrastructure outage is distinguished from a student compilation or execution failure. Assignment enrollment, instructor policy, and due-date checks precede execution for course work. Independent practice is stored separately from assignment context.

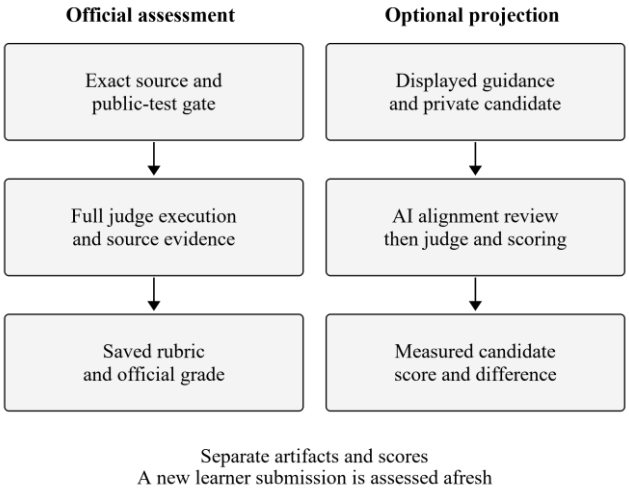


Fig. 1. Separate official assessment and optional candidate projection routes. AI alignment review is distinct from execution verification.

B. Input Contracts and Output Validation

Each problem must define an executable input contract. Human-readable examples containing labels or nested arrays cannot be assumed to be the bytes consumed by standard input. The execution fixtures include a nested-list contract with an outer count followed by a width and values for each row. For the endpoint-sharing Merge Intervals example, four pairs are serialized as an outer count of four, followed by four rows each prefixed by width two. A program that reads only the outer count and then pairs will consume those widths as data. Correct serialization and a matching parser are therefore prerequisites to judging an algorithm.

The execution service writes the requested source into a disposable workspace, compiles it when required, and supplies each serialized case through standard input. Per-case results retain compilation status, exit behavior, standard output, standard error, and resource-limit outcomes. Output comparison first normalizes line endings and trailing whitespace, then supports structurally equal JSON, and finally compares whitespace-delimited tokens. It does not generally sort outputs or accept arbitrary numerical tolerances. Problem authors must use the supported output contract; order-insensitive and floating-point tasks may require dedicated comparators. These restrictions are important when interpreting apparent wrong answers.

Runtime containers use a non-root user, disabled network access, a read-only root filesystem, bounded processes, CPU and memory limits, dropped capabilities, and no-new-privileges. Docker documents the relevant runtime controls [7]. These measures reduce exposure from submitted programs, but container isolation is not a formal security guarantee. Compiler setup, input/output processing, and container overhead must also be separated from an algorithmic complexity claim.

C. Complexity and Style Evidence

The structural analyzer inspects loop bounds, nesting, recognized operations, function structure, and allocation patterns. Tree-sitter supplies syntax structure [8]; syntax alone does not prove asymptotic cost. The platform records estimated time and space expressions, confidence, method, status, and supporting signals. Complete structural results at or above the configured direct threshold, 0.85 by default, can proceed without an optional model review. This threshold is an engineering heuristic, not a calibrated probability of correctness.

For weaker evidence, the pipeline can consider observational runtime growth and an enabled AI review. Sparse or noisy observations must not be presented as controlled asymptotic experiments. The production merge preserves existing structural expressions when an AI estimate disagrees, marks unresolved evidence for review, and permits filling genuinely missing fields under the configured policy. Private target complexities are used only by backend scoring; they are not supplied as answers to the reviewer. Missing or unmappable estimates do not receive invented full marks.

Time and space percentages are obtained by comparing the inferred expression with a curated problem target on a finite scoring ladder. The ten ordered buckets are constant, logarithmic, square-root, linear, linearithmic, quadratic, cubic, fourth-degree-or-higher polynomial, exponential, and factorial. Displayed multivariate expressions are retained, while a deterministic projection maps them onto the scoring ladder. This projection is a grading convention, not a universal

equivalence of complexity functions. Incorrect targets or inappropriate dimension assumptions can bias the mark.

Let the submitted and target ranks be r_s and r_t, indexed from one, and let L be the number of buckets. The penalty is twenty-five times max(0, r_s minus r_t) divided by max(1, L minus r_t). Subtracting it from twenty-five, clipping to zero through twenty-five, and rounding to three decimal places gives component marks. Multiplying those marks by four gives a percentage. Time and space follow this relative-rank rule. Unmappable expressions require review instead of a guessed rank.

Space conventions require particular care. In a function-level snippet, a supplied input array can be excluded from auxiliary storage. In a complete standard-input program, an input-sized container allocated by the submitted program counts under the platform's storage policy. Accordingly, the pilot labels and executable fixtures must record their interface and storage convention rather than silently mixing the two.

Style is evaluated by deterministic source checks covering naming, readability, modularity, and maintainability. Each category contributes at most twenty-five points to a percentage score. The checks remove comments before evaluation, so missing comments alone do not lower the style score. Findings are heuristics rather than expert aesthetic judgments. Instructor similarity groups are review evidence and do not contribute to a student's numeric grade or establish misconduct.

IV. Versioned Scoring Policy

A. Components and Weights

For a valid assessment with N evaluated cases and p accepted cases, observed correctness is defined by (1). Compilation failure yields zero correctness; an unavailable judge or missing usable execution evidence leaves correctness unknown. Passing a finite suite is not a proof for all valid inputs. Case counts weight each evaluated case equally, so test diversity and duplicates affect the interpretation of partial credit.

$$C = 100\,p / N,\ N > 0. \quad (1)$$

Table I gives the default rubric. T, M, and Y denote time, space, and style percentages, respectively; C is observed correctness. The weighted quality score and unrounded final policy are given by (2). Instructors may specify nonnegative question or assignment-context weights totaling one hundred percent. The correctness ceiling remains mandatory. Selected weights and the policy version are stored with the submission and cannot silently change when a question is edited later.

TABLE I. DEFAULT VERSIONED RUBRIC

Component	Weight	Evidence
Correctness	30%	Observed case verdicts
Time	35%	Inferred cost and target rank
Space	25%	Allocation evidence and target rank
Style	10%	Deterministic source findings

$$Q = 0.30C + 0.35T + 0.25M + 0.10Y, \quad (2)$$
$$S_0 = min(Q, C).$$

The allocation gives time and space a combined sixty percent of the weighted score, matching the platform's complexity-focused purpose. Correctness retains thirty percent within that sum and independently limits the outcome. Style receives ten percent so source presentation cannot dominate demonstrated behavior. These are transparent design choices, not values proven optimal by a literature citation. A defensible empirical justification would compare alternatives against independently collected instructor judgments on a held-out set. Such grades were not available for this study.

B. Correctness Ceiling and Rounding

Proposition 1: for finite component percentages between zero and one hundred and nonnegative weights summing to one, the unrounded policy satisfies zero less than or equal to S_0 less than or equal to C less than or equal to one hundred. Proof: the weighted sum Q is nonnegative and bounded by one hundred. Taking its minimum with C preserves nonnegativity and makes the result no larger than C. In particular, zero correctness forces zero marks regardless of the other components. This argument applies to every valid instructor weight configuration.

Ordinary decimal rounding could violate the exact ceiling near one hundred. The implemented display mark therefore uses (3), where the round operator denotes the implementation's nearest-tenth rounding. Taking the smaller value prevents a partial correctness fraction from rounding up past its bound. Regression cases include near-perfect fractions rather than only integer percentages.

$$S = min(round_{0.1}(S_0), \lfloor 10C \rfloor / 10). \quad (3)$$

For example, if forty-four of sixty-one cases pass and all remaining components equal one hundred, Q is approximately 91.64, whereas C is approximately 72.13; the displayed score is 72.1. A fixed seventy-nine-point failure cap is unnecessary. A submission can still have a high partial score when very few cases fail, but its verdict remains not accepted, and failed results are not labelled Excellent. Acceptance and numeric quality are separate outputs.

For fixed time, space, and style scores, increasing correctness cannot decrease the unrounded policy: both C and its weighted contribution are nondecreasing. Improving another component cannot increase the mark while correctness is the active ceiling. The report should explain this plateau rather than promise a gain for every optimization. This property constrains arithmetic behavior; it does not validate the underlying complexity estimates or the pedagogical fairness of the weights.

C. Incomplete Evidence

Unknown components with positive weight remain unknown. The platform reports an incomplete assessment and bounds derived from known contributions and remaining possible contributions; it does not renormalize the other weights. Correctness is required even when an instructor assigns it zero weight, because the ceiling still depends on it. A confirmed zero correctness value already determines a zero final mark. An uncertainty-aware report is preferable to presenting a precise total assembled from unsupported assumptions.

V. Evidence Guided Feedback and Candidate Projection

Feedback prioritizes compilation and observed correctness issues before efficiency or style. The report can display a failed public case or intentionally reveal one failed private case as a learning example. Remaining private case data are redacted from learner responses. This is an explicit disclosure policy, not a claim that every private case remains secret. Repeated submissions may expose additional information; deployment policy must account for this tradeoff.

The default guided workflow asks the learner to implement one scoped change. Stages distinguish foundational repairs, algorithmic correctness, local optimization, data-structure changes, space optimization, and refinement. Guidance is available independently of score projection. It is not a replacement submission, and instructor policies can restrict access to hints or explanations.

A private candidate generator receives the learner's source, public problem context, stage, and the exact displayed guidance. A separate AI-assisted review examines whether the advice is supported by source operations, whether the candidate implements the material actions, and whether it preserves stage scope. Unsupported claims such as inferring cubic time solely from nested containers should be rejected. Missing evidence or an inconclusive review withholds the projection. Because this remains a model judgment, it is not a formal equivalence or complexity proof.

Only an eligible candidate proceeds to execution on all configured cases. Every evaluated case must pass, and the weighted assessment must be complete. Its measured score is recomputed with the original submission's saved rubric. Equation (4) defines the displayed difference. This value belongs to the private candidate, not to whatever code the learner may later write. A new learner submission is executed and assessed afresh; cached projection evidence cannot override its official result.

$$\Delta = S(candidate) - S(saved\ submission). \quad (4)$$

The interface distinguishes checking, slower verification, paused status refresh, measured improvement, verified no increase, and unavailable verification. A completed zero or negative difference is retained as an outcome instead of triggering a fabricated positive gain. An expandable table shows component percentages, weights, and weighted contributions. A separate explicit retry can request fresh verification, while Check again resumes reading saved status without generating another candidate.

Background requests use bounded retries, persisted attempt counts, and request identifiers to avoid stale work replacing a newer stage. Candidate source and private assessment metadata remain server-side. Audit matching includes source fingerprints and evaluation context, including rubric and test configuration. The extra generation and alignment-review steps can increase waiting time; no live-provider latency or success-rate claim is established by the controlled tests below.

VI. Verification Method and Results

A. Frozen Implementation and Research Questions

The implementation snapshot is Git commit d4cdb86, with rubric version complexity-focused-v2 and guidance version six, verified on 30 October 2026. Its source and prompt manifest records SHA-256 values, allowed configuration, Python 3.11.16, Linux x86-64, and twelve reported logical processors. Credentials and database contents are excluded. The evidence separates three questions: whether execution and scoring contracts hold; whether static complexity labels agree with supplied AI proposals; and which findings remain insufficient for human grading or educational claims.

The backend suite passed 333 tests, projection rendering passed nine checks, and frontend lint and build passed. Coverage includes arithmetic, incomplete evidence, privacy, stale requests, alignment rejection, and API behavior. Optional provider responses are mocked where needed. These counts establish regression behavior, not grading accuracy.

B. Live Execution and Score Contracts

Eight live execution checks used original synthetic programs and three inputs per check. Table II summarizes the matrix. Sum programs passed positive, negative, and zero inputs in all five languages. C++ Merge Intervals passed endpoint-sharing, unsorted overlapping, and nested-interval fixtures under the documented row-width contract. A changed incorrect source failed all three cases, and restoring the correct source passed again. Thus all eight checks met their expected outcomes; the intentionally incorrect program is not counted as a successful solution.

TABLE II. LIVE SYNTHETIC EXECUTION CONTRACTS

Check	Runs	Passed cases per run
Sum in five languages	5	3/3 each
Nested-array merge in C++	1	3/3
Changed incorrect source	1	0/3 as expected
Restored correct source	1	3/3
Contracts meeting expectation	8/8	No model accuracy claim

A separate contract used an inefficient summation program and a streaming candidate with an isolated in-memory database and the optional provider disabled. Under default weights, the original scored 85.4 and the candidate scored 97.4. Under a custom twenty/fifty/twenty-five/five allocation, the original scored 81.9 and the candidate scored 98.7. In both configurations, the projected candidate score exactly matched a fresh official assessment of the same candidate, and all three candidate cases passed. The differences were 12.0 and 16.8 points, respectively. This checks shared assessment plumbing under two policies; it does not estimate AI generation reliability or validate semantic guidance alignment on real model outputs.

The scoring audit additionally evaluated twenty-eight synthetic component combinations under the default, equal-weight, and correctness-heavier policies. All eighty-four policy-scenario evaluations obeyed the correctness ceiling. Because that ceiling is shared by the comparators, the result confirms arithmetic consistency but does not identify the best allocation. Regression tests and the proposition provide different evidence: examples exercise the implementation, while the argument establishes the stated policy invariant under its assumptions.

C. AI Assisted Complexity Label Pilot

The pilot comprises twenty original AI-assisted synthetic function snippets: four variants in each of Python, C, C++, Java, and JavaScript. The variants cover direct summation, deliberately repeated summation, linear search, and binary search, belonging to only two original problem groups. Search inputs are sorted ascending arrays. Function inputs are supplied by the caller; the review convention is worst-case time and auxiliary space excluding those inputs, with unit-cost arithmetic assumptions. The snippets are not a representative sample of student programs or twenty independent problems.

Two externally supplied files proposed labels for every canonical solution identifier. Model identities, settings, prompts, and independence were not verified. Their provenance remains AI-assisted and unverified. We preserved the original files, used canonical source hashes and problem groups, and recorded returned metadata discrepancies. Neither file supplies human expert reviews or instructor grades.

With the optional provider disabled for static comparison, both proposed label sets agreed on all twenty time labels and all twenty space labels. The analyzer returned an analyzed result for all twenty snippets. It matched nineteen time labels, twenty space labels, and nineteen joint labels. Table III reports exact counts by language. Overall joint exact label agreement was ninety-five percent with full coverage on this particular pilot. This is agreement with model proposals, not expert-ground-truth accuracy or application-wide accuracy.

TABLE III. EXACT AGREEMENT WITH AI LABEL PROPOSALS

Language	n	Time	Space	Joint
Python	4	3/4	4/4	3/4
C	4	4/4	4/4	4/4
C++	4	4/4	4/4	4/4
Java	4	4/4	4/4	4/4
JavaScript	4	4/4	4/4	4/4
Total	20	19/20	20/20	19/20

The mismatch is informative. A Python program repeatedly sums the same array inside a loop over that array's length. Both bounds denote the same size, so its work is quadratic under the stated convention. The analyzer reports a product of two symbolic bounds instead of consolidating them into one squared bound. Those expressions specialize to the same growth when the symbols are equal, but the strict comparison retains the mismatch. Global normalization of every two-variable product to a squared single-variable expression would hide valid distinctions and was not applied to improve the reported count.

We report descriptive counts rather than inferential confidence intervals: translated variants and related approaches violate a simple independent-sample interpretation. No model-only versus hybrid superiority claim is made, because controlled provider predictions and independently adjudicated labels were not available. The static-only comparison is a reproducible baseline for a subsequent broader study.

D. Controlled Comparison of Correctness Constraints

To isolate the correctness constraint from weight selection, we hold component vectors and weights fixed across four rules: weighted-only Q, a fixed cap min(Q, 79) when C is below one hundred, multiplicative scaling Q times C divided by one hundred, and min(Q, C). We reuse the twenty-eight-scenario audit grid: seven correctness levels (zero, twenty-five, fifty, seventy-five, ninety, ninety-nine, and one hundred), crossed with four time/space/style triples: 100/100/100, 50/100/80, 100/50/80, and 50/50/80. Three C/T/M/Y profiles (30/35/25/10, 25/25/25/25, and 40/30/20/10) produce eighty-four configurations per rule. This is a synthetic mechanism experiment, not sampled instructor grades.

The violation metric is an unrounded mark greater than C, with a numerical tolerance of one billionth. Table IV shows forty-nine violations for weighted-only aggregation and forty-three for the fixed cap. Both also award nonzero marks in all twelve zero-correctness configurations. Multiplication and the ceiling have no violations. The comparator uses the current weights to isolate the cap's effect; it does not reproduce all historical application settings. The reproducible experiment is frozen at commit 7ddc4aa, while the application snapshot remains d4cdb86.

TABLE IV. CONTROLLED SCORING RULE COMPARISON

Rule	Mark > C (of 84)	Nonzero at C = 0 (of 12)
Weighted only	49	12
Fixed cap 79	43	12
Multiplicative scaling	0	0
Correctness ceiling	0	0

Multiplication also respects the ceiling, because Q and C are each at most one hundred. It is no greater than either operand, and therefore no greater than min(Q, C). The proposed policy selects the largest mark compatible with both the weighted total and the stipulated correctness bound; it avoids another whole-score correctness discount. This is a declared partial-credit preference, not proof that multiplication is educationally inferior.

With default weights and perfect remaining components, increasing C from ninety-nine to one hundred moves the fixed-cap mark from 79 to 100, whereas the ceiling moves from 99 to 100. Separately, at correctness 99.999, ordinary nearest-tenth rounding can display 100 even for min(Q, C); conservative rounding displays 99.9. We separate this presentation check from the unrounded comparison to avoid attributing a rounding difference to the constraint itself.

Weights still matter. With complete correctness, time and space percentages of fifty, and style eighty, the default gives 68.0, equal weighting 70.0, and correctness-heavier weighting 73.0. Higher totals do not establish a better rubric. For correctness weight w below one and remaining weighted contribution B, the ceiling is active when C is no larger than B divided by one minus w. Reducing correctness's internal weight can therefore change both grades and the region where optimization becomes visible.

For future label studies, coverage should be the number of scoreable analyzer answers divided by the number of eligible

programs. Exact agreement conditional on answering should use the answered-program denominator, with time, space, and joint counts reported separately. In this pilot, those denominators coincide at twenty; they will not coincide when the analyzer abstains. Keeping both metrics prevents a selective system from appearing accurate by silently omitting difficult programs.

E. Reproducibility and Data Availability

The tagged artifact release [9] provides twenty canonical source fixtures, normalized AI label proposals, provenance hashes, execution reports, frozen-snapshot test transcripts, and reproduction instructions. The tags paper-implementation-20261030 and paper-scoring-experiment-20261030 identify the cited implementation and comparison commits. Proposal aliases are not authenticated reviewer identities; raw review prose is excluded. Published labels retain AI-assisted unverified provenance. The release contains no learner database, credentials, or real private course cases. It supports reproduction of the reported synthetic results, not independent validation of the proposed labels.

VII. Discussion and Limitations

The principal result is consistency between explicit policy and observed implementation behavior. The correctness ceiling prevents efficiency and style from lifting the mark above observed correctness, and the synthetic candidate contract confirms shared scoring in two configurations. Neither result proves that the complexity inference, test suite, or default weights are educationally optimal. Transparency makes a policy inspectable; it does not remove judgment from assessment design.

Several threats constrain interpretation. Test-case correctness depends on specification quality, input serialization, comparator semantics, and coverage of boundary cases. The finite complexity ladder compresses multivariate and library-dependent costs. Confidence is heuristic, and runtime observations can reflect compilation, scheduling, or container overhead. The pilot has two problem groups, related cross-language variants, AI-proposed labels, and no held-out test set. The remaining symbolic-bound discrepancy is retained as a limitation rather than repaired on the reporting set.

Feedback quality and alignment review remain unvalidated by independent experts. A candidate can pass finite tests yet fail on untested inputs, and a model can approve an unsupported explanation. The guard restricts projection eligibility without guaranteeing a learning outcome. Live-provider success rate and representative latency have not been benchmarked.

Before broader efficacy claims, two qualified reviewers should label a diverse corpus and adjudicate disagreements with problem-level holdouts. Blinded instructor grades would permit comparisons of policy error and agreement. Feedback should be rated for factuality, relevance, actionability, and private-data disclosure. Repeated warm and cold requests should report latency distributions and provider failures.

The reported evaluation uses synthetic fixtures and supplied AI review artifacts; it does not analyze identifiable learner records or report participant outcomes. No human-study ethics approval is asserted. Imported public problem collections are not treated as a publication dataset with assumed redistribution rights. A reproducibility package should include only fixtures and artifacts for which the authors have permission, and should exclude credentials, private course data, and real judge secrets. Similarity evidence is not proof of plagiarism, and this work does not claim plagiarism-screening clearance.

VIII. Conclusion

CodeVedha links execution evidence, resource estimates, style analysis, and optional AI guidance through an auditable assessment contract. Controlled comparison shows that a fixed failure cap does not enforce the chosen correctness bound. Both multiplication and a ceiling enforce it, while the ceiling preserves more partial credit under that declared policy. Execution contracts and candidate-score consistency checks support the implementation, and an explicitly AI-assisted pilot exposes a symbolic-bound limitation. Independent expert labels, instructor-grade validation, real-provider evaluation, and learning studies remain necessary for broader accuracy or educational claims.

Acknowledgment

OpenAI Codex was used to draft and revise the abstract and Sections I–VIII, assist with verification scripts and synthetic fixtures, and generate the architecture illustration. The pilot label proposals were supplied as external AI-assisted artifacts, with unverified self-reported model identities. They are reported as AI proposals, not human expert judgments. Authors are responsible for checking the final text, references, results, and disclosures before submission.

References

[1] C. Douce, D. Livingstone, and J. Orwell, “Automatic test-based assessment of programming: A review,” Journal on Educational Resources in Computing, vol. 5, no. 3, Art. no. 4, 2005, doi: 10.1145/1163405.1163409.

[2] H. Keuning, J. Jeuring, and B. Heeren, “A systematic literature review of automated feedback generation for programming exercises,” ACM Transactions on Computing Education, vol. 19, no. 1, Art. no. 3, 2018, doi: 10.1145/3231711.

[3] C. Piech, J. Huang, A. Nguyen, M. Phulsuksombati, M. Sahami, and L. Guibas, “Learning program embeddings to propagate feedback on student code,” in Proceedings of the 32nd International Conference on Machine Learning, vol. 37, 2015, pp. 1093–1102.

[4] V. Sahu, G. R. Gupta, R. Borikar, and N. Mane, “Autograder+: A multi-faceted AI framework for rich pedagogical feedback in programming education,” arXiv:2510.26402, 2025, doi: 10.48550/arXiv.2510.26402.

[5] D. Hendrycks et al., “Measuring coding challenge competence with APPS,” in Proceedings of the NeurIPS Datasets and Benchmarks Track, 2021, arXiv:2105.09938.

[6] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[7] Docker, Inc., “docker container run,” Docker Documentation. Accessed: Oct. 30, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/run/

[8] Tree-sitter Contributors, “Tree-sitter: An incremental parsing system for programming tools.” Accessed: Oct. 30, 2026. [Online]. Available: https://github.com/tree-sitter/tree-sitter

[9] CodeVedha Project, “Paper evaluation artifacts and reproduction guide,” CodeAssessment repository, release paper-artifacts-20261030, 2026. Accessed: Oct. 30, 2026. [Online]. Available: https://github.com/multiagentcodereview2026/CodeAssessment/tree/paper-artifacts-20261030/docs/research-paper/artifacts